

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
Факультет физико-математических и естественных наук
Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 12

дисциплина: Архитектура компьютера

<<Программирование в командном процессоре ОС UNIX. Ветвления и циклы>>

Студент: ИБРАХИМ ХИССЕИН ГАНА
Группа: НПИбд 01–25

МОСКВА
2026 г.

Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Выполнение работы

1. Скрипт с `getopts` (поиск строк в файле)

Файл `search.sh`:

```
``bash
#!/bin/bash
input=""
output=""
pattern=""
case_sensitive=""
line_numbers=""

while getopts ":i:o:p:Cn" opt; do
    case $opt in
        i) input="$OPTARG" ;;
        o) output="$OPTARG" ;;
        p) pattern="$OPTARG" ;;
        C) case_sensitive="-i" ;;
        n) line_numbers="-n" ;;
        \?) echo "Option invalide: -$OPTARG" >&2; exit 1 ;;
        :) echo "L'option -$OPTARG requiert un argument." >&2; exit 1 ;;
    esac
done

if [ -z "$input" ] || [ -z "$pattern" ]; then
    echo "Usage: $0 -i fichier_entree -p motif [-o fichier_sortie] [-C] [-n]"
    exit 1
fi

if [ -n "$case_sensitive" ]; then
    grep_cmd="grep $case_sensitive $line_numbers"
else
    grep_cmd="grep $line_numbers"
fi

if [ -n "$output" ]; then
    $grep_cmd "$pattern" "$input" > "$output"
    echo "Résultats écrits dans $output"
else
    $grep_cmd "$pattern" "$input"
fi
```

```

ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~$ mkdir -p ~/lab13
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~$ cd ~/lab13
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ nano search.sh
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ chmod +x search.sh
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ echo "Hello world" > test.txt
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ echo "hello again" >> test.txt
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ ./search.sh -i test.txt -p world
Hello world
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ ./search.sh -i test.txt -p hello -C
Hello world
hello again
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ █

```

```

echo "Hello world" > test.txt
echo "hello again" >> test.txt
./search.sh -i test.txt -p world
./search.sh -i test.txt -p hello -C

```

```

GNU nano 7.2                                check_num.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("Entrez un nombre entier: ");
    scanf("%d", &n);
    if (n > 0)
        exit(1);
    else if (n < 0)
        exit(2);
    else
        exit(0);
}

```

2. Programme C + script bash (code de retour)

2.1. Programme C (check_num.c)

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("Entrez un nombre entier: ");
    scanf("%d", &n);
    if (n > 0) exit(1);
    else if (n < 0) exit(2);
    else exit(0);
}

```

```

GNU nano 7.2                                test_num.sh *
#!/bin/bash
./check_num
case $? in
  0) echo "Le nombre est nul" ;;
  1) echo "Le nombre est positif" ;;
  2) echo "Le nombre est négatif" ;;
esac

```

2.2. Script bash (test_num.sh)

```

#!/bin/bash
./check_num
case $? in
  0) echo "Le nombre est nul" ;;
  1) echo "Le nombre est positif" ;;
  2) echo "Le nombre est négatif" ;;
esac

```

```

ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ nano check_num.c
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ gcc check_num.c -o check_num
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ nano check_num.c
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ nano test_num.sh
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ chmod +x test_num.sh
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ ./test_num.sh
Entrez un nombre entier: 58
Le nombre est positif
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ 

```

Compilation et test:

```

gcc check_num.c -o check_num
./test_num.sh

```

```
GNU nano 7.2                                files.sh *
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <N> (créer N fichiers .tmp)"
    echo "Pour supprimer: $0 clean"
    exit 1
fi

if [ "$1" = "clean" ]; then
    rm -f [0-9]*.tmp
    echo "Fichiers .tmp supprimés."
    exit 0
fi

N=$1
for ((i=1; i<=N; i++)); do
    touch "$i.tmp"
done
echo "$N fichiers créés :"
ls -l [0-9]*.tmp 2>/dev/null || echo "Aucun fichier .tmp trouvé."
```

3. Création / suppression de fichiers temporaires (files.sh

```
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <N> (créer N fichiers .tmp)"
    echo "Pour supprimer: $0 clean"
    exit 1
fi

if [ "$1" = "clean" ]; then
    rm -f [0-9]*.tmp
    echo "Fichiers .tmp supprimés."
    exit 0
fi

N=$1
for ((i=1; i<=N; i++)); do
    touch "$i.tmp"
done
echo "$N fichiers créés :"
ls -l [0-9]*.tmp 2>/dev/null || echo "Aucun fichier .tmp trouvé."
```

```

ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ chmod +x files.sh
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ ./files.sh 5
5 fichiers créés :
-rw-rw-r-- 1 ibrahim ibrahim 0 mai  9 23:28 1.tmp
-rw-rw-r-- 1 ibrahim ibrahim 0 mai  9 23:28 2.tmp
-rw-rw-r-- 1 ibrahim ibrahim 0 mai  9 23:28 3.tmp
-rw-rw-r-- 1 ibrahim ibrahim 0 mai  9 23:28 4.tmp
-rw-rw-r-- 1 ibrahim ibrahim 0 mai  9 23:28 5.tmp
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ ./files.sh clean
Fichiers .tmp supprimés.
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ █

```

Tests:

```

chmod +x files.sh
./files.sh 5
./files.sh clean

```

```

GNU nano 7.2                                backup_recent.sh *
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <répertoire>"
    exit 1
fi

dir="$1"
archive="backup_$(basename "$dir")_$(date +%Y%m%d).tar.gz"
find "$dir" -type f -mtime -7 -print0 | tar -czvf "$archive" --null -T -
echo "Archive créée : $archive"█

```

4. Archivage des fichiers modifiés récemment (backup_recent.sh)

```

#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <répertoire>"
    exit 1
fi

dir="$1"
archive="backup_$(basename "$dir")_$(date +%Y%m%d).tar.gz"
find "$dir" -type f -mtime -7 -print0 | tar -czvf "$archive" --null -T -
echo "Archive créée : $archive"

```

```

ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ chmod +x backup_recent.sh
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ mkdir -p testdir
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ touch testdir/oldfile
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ touch -d "2 days ago" testdir/recentfile
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ ./backup_recent.sh testdir
testdir/recentfile
testdir/oldfile
Archive créée : backup_testdir_20260509.tar.gz
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ ls -l backup_*.tar.gz
-rw-rw-r-- 1 ibrahim ibrahim 150 mai  9 23:30 backup_testdir_20260509.tar.gz
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ █

```

```

mkdir -p testdir
touch testdir/oldfile
touch -d "2 days ago" testdir/recentfile
./backup_recent.sh testdir
ls -l backup_*.tar.gz

```

```

ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ chmod +x backup_recent.sh
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ mkdir -p testdir
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ touch testdir/oldfile
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ touch -d "2 days ago" testdir/recentfile
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ ./backup_recent.sh testdir
testdir/recentfile
testdir/oldfile
Archive créée : backup_testdir_20260509.tar.gz
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ ls -l backup_*.tar.gz
-rw-rw-r-- 1 ibrahim ibrahim 150 mai  9 23:30 backup_testdir_20260509.tar.gz
ibrahim@ibrahim-IdeaPad-5-Pro-14ITL6:~/lab13$ █

```

Выводы

В ходе работы были разработаны четыре скрипта:

Анализ командной строки с getoptс и поиск строк.

Взаимодействие с программой на C через код возврата.

Создание и удаление большого количества временных файлов.

Архивация только недавно изменённых файлов с помощью find.

Получены навыки использования getoptс, обработки кодов возврата, циклов и условных операторов в bash.

Ответы на контрольные вопросы

1. Каково предназначение команды getoptс?

getopts анализирует опции командной строки в скриптах bash, позволяя легко обрабатывать короткие опции (например, -i, -o) и их аргументы.

2. Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы (*, ?, []) используются для шаблонов имён файлов (globbing). Например, *.txt соответствует всем файлам с расширением .txt.

3. Какие операторы управления действиями вы знаете?

if/then/else, case, for, while, until, break, continue, exit.

4. Какие операторы используются для прерывания цикла?

break – выход из цикла; continue – переход к следующей итерации.

5. Для чего нужны команды false и true?

true возвращает код 0 (успех), false возвращает ненулевой код (ошибка). Используются для организации бесконечных циклов или логических проверок.

6. Что означает строка if test -f man\\${s}/\\${i}.\\${s}, встреченная в командном файле?

Проверяет, существует ли файл man\\${s}/\\${i}.\\${s} (значения переменных \${s} и \${i} подставляются). Обратный слеш экранирует \$, чтобы не интерпретировать переменную внутри кавычек.

7. Объясните различие между конструкциями while и until.

while выполняет блок пока условие истинно (код возврата 0).

until выполняет блок до тех пор, пока условие не станет истинным (повторяет, пока ложно).